Model Compression

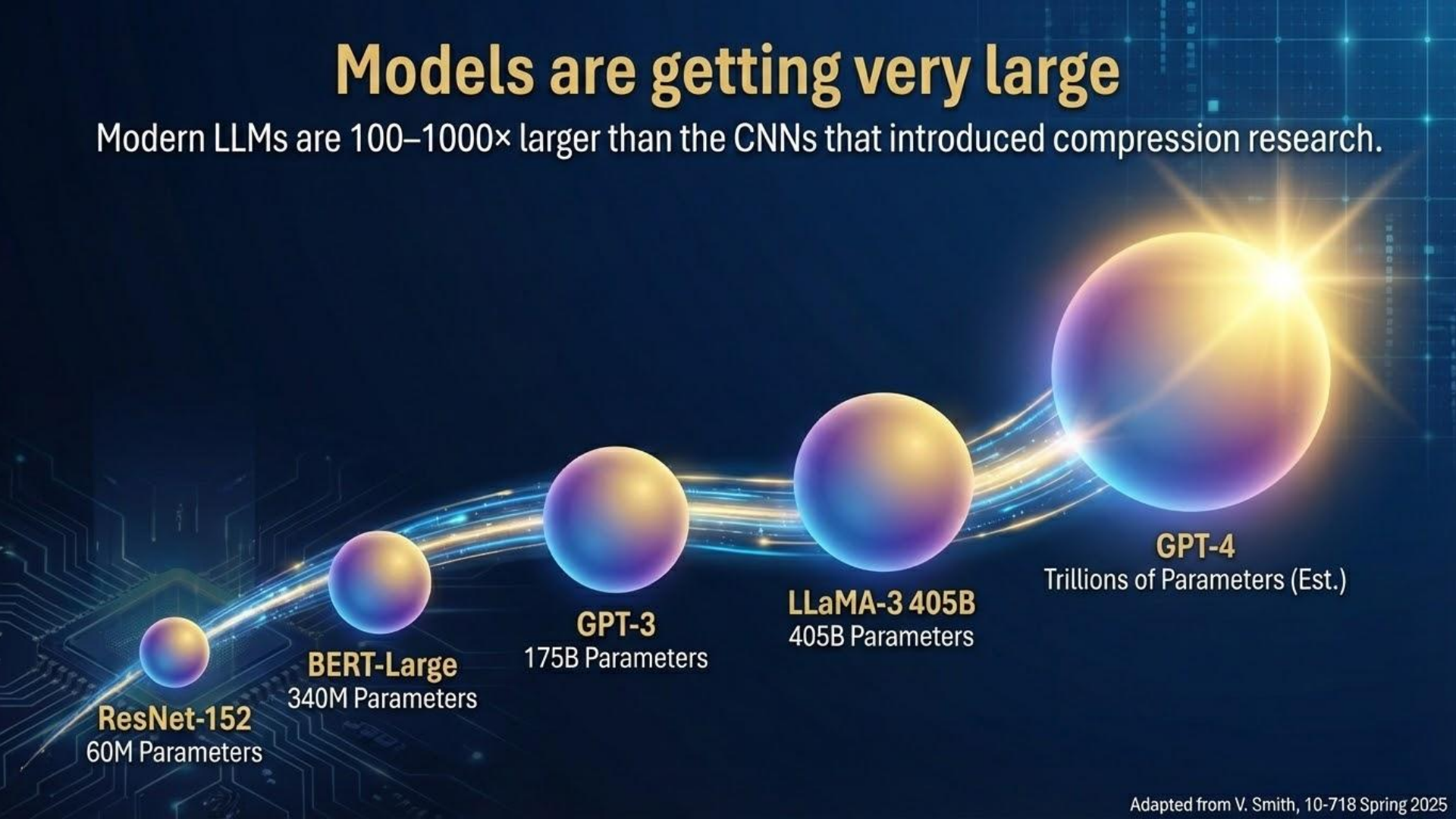


10-718: Machine Learning in Practice · CMU · Spring 2026

Steven Kolawole | Language Technologies Institute

Models are getting very large

Modern LLMs are 100–1000× larger than the CNNs that introduced compression research.



Inference: why should we care?

Train once, infer many times — production ML systems mostly just do inference.

Where do we run inference?

Cloud (AWS, GCP)

Concerns: latency, cost

User devices (phone, laptop)

Concerns: memory, model size

IoT (wearables, sensors)

Concerns: energy, memory

What do we measure?

Accuracy	How good are predictions?
Latency	Time per prediction
Throughput	Predictions per second
Model size	Bytes to store/transmit
Energy	Power per prediction
Cost	$/query at scale

Adapted from V. Smith, 10-718 Spring 2025

Model compression

Goal: find a smaller, cheaper model with the same (or similar) accuracy.

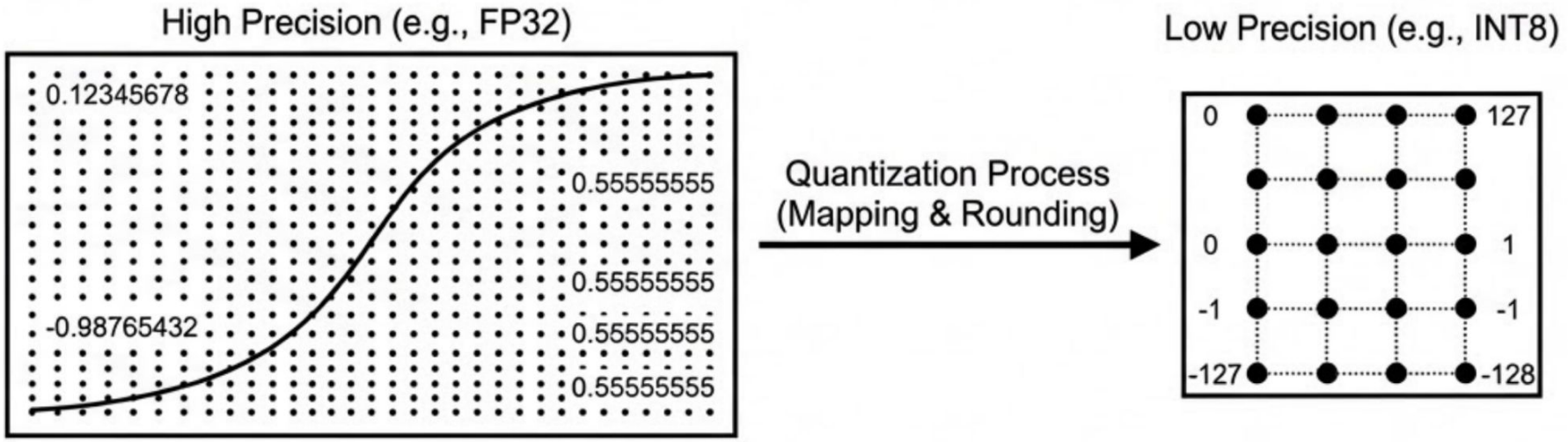
Four main families of techniques

1.	**Quantization**	Reduce numerical precision of weights/activations (FP32 → INT8, INT4)
2.	**Pruning**	Remove low-importance weights or entire structures (heads, layers)
3.	**Knowledge distillation**	Train a smaller student model to mimic a large teacher model
4.	**Efficient architectures**	Design models to be compact from the start (MobileNet, ShuffleNet)

Combining these methods is common in production.

Quantization

Use low-precision arithmetic at inference time.



Quantization

Use low-precision arithmetic at inference time.

The precision ladder

Format	Bits	Memory	Notes
FP32	32-bit	1×	Baseline — all hardware
FP16	16-bit	0.5×	Default serving on modern GPUs
INT8	8-bit	0.25×	PTQ usually works; small quality cost
INT4	4-bit	0.125×	Active research: GPTQ, AWQ, etc.

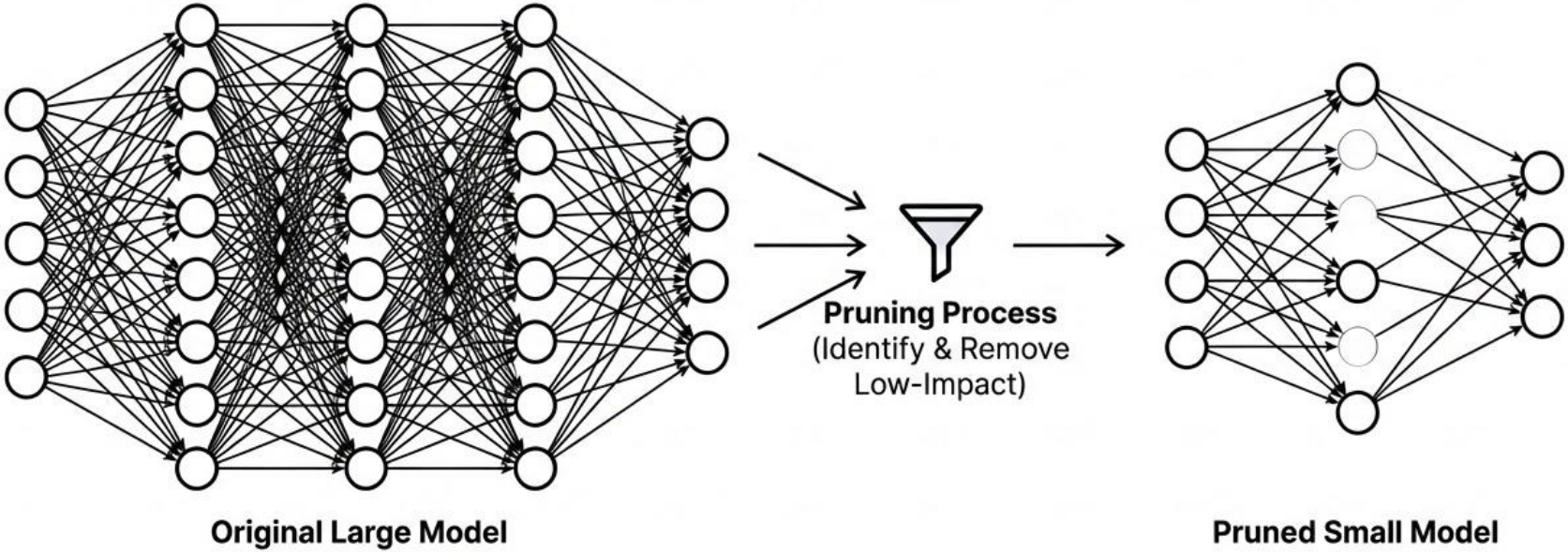
PTQ vs. QAT

Post-Training Quantization (PTQ)
Quantize after training. No retraining needed. Fast and cheap. Works well at INT8; degrades at INT4 without careful calibration.

Quantization-Aware Training (QAT)
Simulate low precision during training. Better quality at INT4 — but requires a full training run.

Pruning

Remove low-impact weights or structures to create a smaller model.



Pruning

Remove low-impact weights or structures to create a smaller model.

What to prune?

Unstructured

Zero out individual weights. Can reach very high sparsity (90%+) but sparse matrices are hard to accelerate on real hardware.

Structured

Remove entire attention heads, MLP dimensions, or layers. Produces a dense, smaller model — real speedup on standard hardware.

How to rank importance?

Magnitude · gradient sensitivity · Taylor expansion · activation statistics

When to prune?

One-shot

Prune once after training. Simple but degrades more.

Iterative

Prune → fine-tune → repeat. Better quality, more expensive.

During training

e.g., dropout. Regularization-style approach.

Knowledge distillation

Train a small student model to mimic the output of a large teacher model.

Key idea

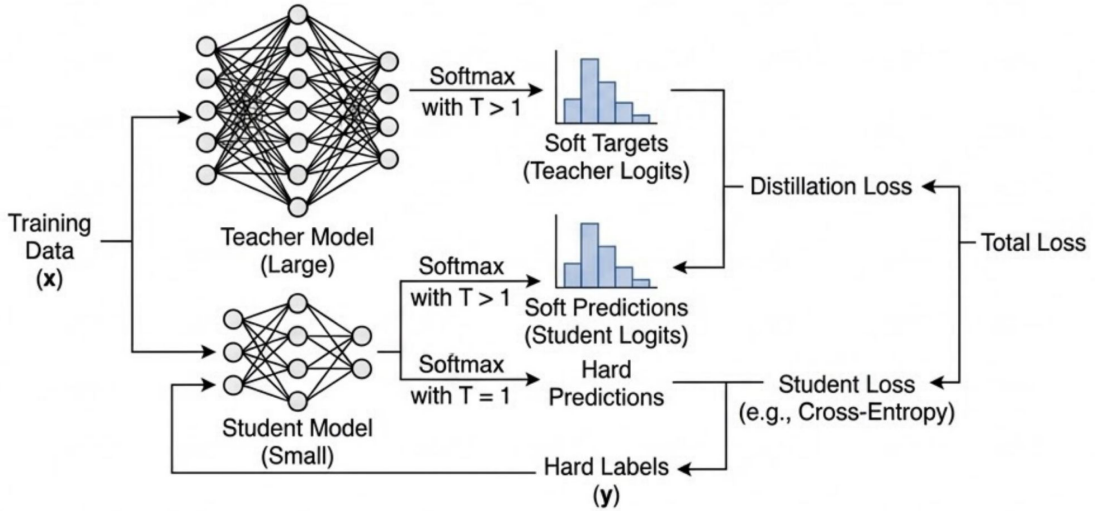
The teacher's soft output distribution (logits) is richer than hard labels — it encodes similarity structure between classes. A student trained on these soft targets learns more than one trained on one-hot labels.

Temperature parameter T

Softmax with T > 1 softens the distribution, amplifying the signal in small probabilities. Higher T → smoother targets → more transferable knowledge.

Advantage over pruning/quantization

The student architecture can differ entirely from the teacher — you are not modifying the original model, you are training a new one.



Hinton et al., 2015; Bucila et al., 2006

Efficient architectures

An alternative: design models to be efficient from the start, rather than compressing post-hoc.

- Architectures like MobileNet and ShuffleNet use depthwise separable convolutions and sparse connectivity to reduce FLOPs and parameters.
- For efficiency, start by training one of these networks — in contrast to training a large model and then compressing it.
- Neural architecture search (NAS) can automate finding efficient designs, pushing the Pareto frontier between accuracy and cost.

Table 8. MobileNet Comparison to Popular Models

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogleNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138

Howard et al., 2017

A word of caution

Evaluating compression claims

There are two metrics we care about: accuracy and cost. The right question is whether a method is on the Pareto frontier.

The standard ML paper template for compression

- Describe a lossy compression technique
- Apply it to a big, redundant model
- Apply one technique uniformly
- Evaluation: "I lost only X in accuracy, but gained Y in cost!" (X sounds small, Y sounds big)

The right standard: Pareto optimality

A model is Pareto optimal if there is no other model achieving the same (or higher) accuracy at the same (or lower) cost. Comparisons should be made at the same operating point on the frontier — not cherry-picked.

Adapted from V. Smith, 10-718 Spring 2025 ·

MLSys 2019 keynote: Ofer Dekel (MSR)

These ideas travel…

Quantization	activations (SmoothQuant), KV cache (FP8/INT* cache entries for long context), gradients during training (mixed-precision), embeddings
Pruning	attention heads, entire transformer layers, KV cache entries (StreamingLLM, H20), tokens (early exit, speculating decoding adjacent ideas), experts in MoE models
Distillation	datasets (data distillation / data condensation – compress 1M examples into 1K synthetic ones), reasoning traces (CoT distillation), intermediate representations (feature-level distillation, not just output logits)

Underlying question is always the same!

“what can we remove or approximate without losing what matters?”

What changes at LLM scale?

— Scale amplifies silent failures

A 1% accuracy drop on ImageNet is fine. On a general-purpose LLM, the same compression might silently break code generation, math, or multilingual performance while headline benchmarks look unchanged.

— Calibration data matters enormously

PTQ for large models requires representative calibration data to set quantization ranges. The wrong calibration set quietly degrades out-of-distribution inputs — with no obvious signal.

— The KV cache is a compression target

At long contexts, the KV cache dominates memory; not the weights. Quantizing or evicting cached keys/values is a distinct compression direction your weights-only framing misses.

— MoEs break standard compression assumptions

Experts are sparsely and input-dependently activated — so importance scores computed on a calibration set may not reflect true expert utility.

— Structured pruning is harder without gradients

Removing transformer layers or heads from a 70B model without costly retraining is hard. Standard gradient-based importance scores require 2–3× the memory of the model itself.

Structured pruning of LLMs with only forward passes

Everybody Prune Now: Structured Pruning of LLMs with Only Forward Passes

Steven Kolawole* & **Lucio Dery*** *{skolawol, ldery}@cs.cmu.edu*
Carnegie Mellon University

Jean-François Kagy *jkagy@google.com*
Google Research

Virginia Smith, Graham Neubig & Ameet Talwalkar *{smithv, gneubig, talwalkar}@cmu.edu*
Carnegie Mellon University

Abstract

The problem. Gradient-based structured pruning methods require 2–3× the model's memory for backward passes — making it infeasible to prune large models on commodity hardware.

The key insight. You don't need gradients to measure importance. Bonsai estimates module importance using only forward passes: perturb the model, evaluate sub-models, fit a linear model of importance scores.

Broader point: the memory constraint is itself a design problem. Reformulating around it — not just optimizing within it — opened up hardware accessibility as a first-class concern.

Sit with these for a moment…

01

If distillation produces better quality than pruning at the same compression ratio, why isn't it always the right choice in practice?

→ *Think about deployment timelines, data availability, and retraining cost.*

02

Top-line accuracy barely changes after pruning or quantization. What does that hide — and for whom?

→ *Think about who lives in the long tail of your data distribution.*

03

Compression decisions in production are made by engineers under deadline pressure, not researchers. What should that mean for how we design methods and tooling?

→ *The Hohman et al. paper has a lot to say here.*